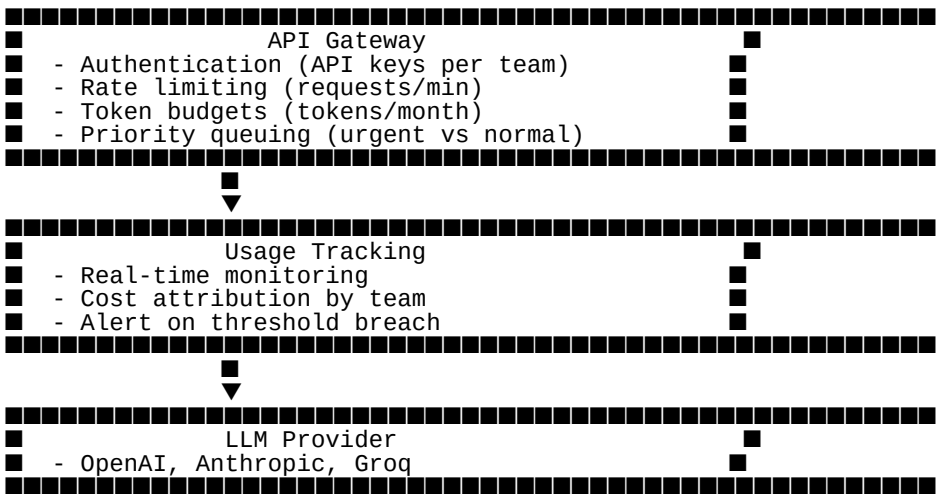
LLM Usage Governance: How to Limit Usage Across Teams

Interview Answer Framework

Short Answer:
"I implement LLM usage governance through three layers: (1) API Gateway with rate limiting and quotas, (2) Cost tracking and budgets per team, and (3) Usage policies with tiered access. This includes technical controls (rate limits, token budgets), monitoring (dashboards, alerts), and organizational policies (approval workflows, priority tiers)."

Complete Solution Architecture



1. Technical Implementation

A. API Gateway with Rate Limiting

```
from flask import Flask, request, jsonify
from functools import wraps
import redis
import time
from datetime import datetime
import hashlib

app = Flask(__name__)
redis_client = redis.Redis(host='localhost', port=6379, decode_responses=True)

# Team quotas configuration
TEAM_QUOTAS = {
```

```
    'engineering': {
        'requests_per_minute': 100,
        'requests_per_day': 10000,
        'tokens_per_month': 10_000_000,
        'max_tokens_per_request': 4000,
        'priority': 'high'
    },
    'product': {
        'requests_per_minute': 50,
        'requests_per_day': 5000,
        'tokens_per_month': 5_000_000,
        'max_tokens_per_request': 2000,
        'priority': 'medium'
    },
    'marketing': {
        'requests_per_minute': 20,
        'requests_per_day': 1000,
        'tokens_per_month': 1_000_000,
        'max_tokens_per_request': 1000,
        'priority': 'low'
    }
}

def get_team_from_api_key(api_key):
    """Extract team from API key"""
    # In production, lookup from database
    key_to_team = {
        'eng_key_123': 'engineering',
        'prod_key_456': 'product',
        'mkt_key_789': 'marketing'
    }
    return key_to_team.get(api_key)

def check_rate_limit(team):
    """Check if team is within rate limits"""
    quota = TEAM_QUOTAS.get(team)
    if not quota:
        return False, "Team not found"

    # Per-minute rate limit
    minute_key = f"rate_limit:{team}:minute:{int(time.time() / 60)}"
    current_minute = redis_client.incr(minute_key)
    redis_client.expire(minute_key, 60)

    if current_minute > quota['requests_per_minute']:
        return False, f"Rate limit exceeded: {quota['requests_per_minute']}/min"

    # Per-day rate limit
    day_key = f"rate_limit:{team}:day:{datetime.now().strftime('%Y-%m-%d')}"
    current_day = redis_client.incr(day_key)
    redis_client.expire(day_key, 86400)

    if current_day > quota['requests_per_day']:
        return False, f"Daily limit exceeded: {quota['requests_per_day']}/day"

    # Monthly token budget
    month_key = f"tokens:{team}:month:{datetime.now().strftime('%Y-%m')}"
    current_tokens = int(redis_client.get(month_key) or 0)

    if current_tokens >= quota['tokens_per_month']:
        return False, f"Monthly token budget exceeded: {quota['tokens_per_month']}"

    return True, "OK"

def track_usage(team, tokens_used, cost):
    """Track team usage"""
    # Token usage
    month_key = f"tokens:{team}:month:{datetime.now().strftime('%Y-%m')}"
    redis_client.incrby(month_key, tokens_used)
    redis_client.expire(month_key, 86400 * 31)

    # Cost tracking
    cost_key = f"cost:{team}:month:{datetime.now().strftime('%Y-%m')}"
    redis_client.incrbyfloat(cost_key, cost)
    redis_client.expire(cost_key, 86400 * 31)

def require_api_key(f):
```

```
    """Decorator to enforce API key and rate limits"""
    @wraps(f)
    def decorated_function(*args, **kwargs):
        api_key = request.headers.get('X-API-Key')

        if not api_key:
            return jsonify({'error': 'API key required'}), 401

        team = get_team_from_api_key(api_key)
        if not team:
            return jsonify({'error': 'Invalid API key'}), 401

        # Check rate limits
        allowed, message = check_rate_limit(team)
        if not allowed:
            return jsonify({
                'error': 'Rate limit exceeded',
                'message': message,
                'team': team
            }), 429

        # Add team to request context
        request.team = team
        return f(*args, **kwargs)

    return decorated_function

@app.route('/api/v1/completions', methods=['POST'])
@require_api_key
def completions():
    """LLM completion endpoint with rate limiting"""
    team = request.team
    quota = TEAM_QUOTAS[team]
    data = request.json

    # Validate request
    max_tokens = data.get('max_tokens', 1000)
    if max_tokens > quota['max_tokens_per_request']:
        return jsonify({
            'error': 'Token limit exceeded',
            'max_allowed': quota['max_tokens_per_request']
        }), 400

    # Call actual LLM (OpenAI, Anthropic, etc.)
    try:
        response = call_llm(data)

        # Track usage
        tokens_used = response['usage']['total_tokens']
        cost = calculate_cost(tokens_used)
        track_usage(team, tokens_used, cost)

        # Add usage info to response
        response['usage_info'] = get_usage_info(team)

        return jsonify(response)

    except Exception as e:
        return jsonify({'error': str(e)}), 500

def call_llm(data):
    """Call actual LLM provider"""
    # Simplified - actual implementation would call OpenAI/Anthropic
    return {
        'choices': [{'message': {'content': 'Response'}}],
        'usage': {'total_tokens': 150}
    }

def calculate_cost(tokens):
    """Calculate cost based on tokens"""
    # GPT-4: $0.03 per 1K input tokens, $0.06 per 1K output tokens
    # Simplified: average $0.045 per 1K tokens
    return (tokens / 1000) * 0.045

def get_usage_info(team):
    """Get current usage for team"""
    month_key = f"tokens:{team}:month:{datetime.now().strftime('%Y-%m')}"
```

```
    cost_key = f"cost:{team}:month:{datetime.now().strftime('%Y-%m')}"

    tokens_used = int(redis_client.get(month_key) or 0)
    cost_used = float(redis_client.get(cost_key) or 0)
    quota = TEAM_QUOTAS[team]

    return {
        'team': team,
        'tokens_used': tokens_used,
        'tokens_limit': quota['tokens_per_month'],
        'tokens_remaining': quota['tokens_per_month'] - tokens_used,
        'usage_percent': (tokens_used / quota['tokens_per_month']) * 100,
        'cost_used': round(cost_used, 2),
        'requests_per_minute': quota['requests_per_minute']
    }

@app.route('/api/v1/usage', methods=['GET'])
@require_api_key
def usage():
    """Get usage statistics for team"""
    team = request.team
    return jsonify(get_usage_info(team))

if __name__ == '__main__':
    app.run(debug=True, port=8000)
```

Usage:

```
# Team makes request
curl -X POST http://localhost:8000/api/v1/completions \
  -H "X-API-Key: eng_key_123" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-4",
    "messages": [{"role": "user", "content": "Hello"}],
    "max_tokens": 100
  }'

# Response includes usage info
{
  "choices": [...],
  "usage_info": {
    "team": "engineering",
    "tokens_used": 1250000,
    "tokens_limit": 10000000,
    "tokens_remaining": 8750000,
    "usage_percent": 12.5,
    "cost_used": 56.25
  }
}
```

B. Token Budget System

```
class TokenBudgetManager:
    """Manage token budgets per team"""

    def __init__(self, redis_client):
        self.redis = redis_client

    def allocate_budget(self, team, monthly_tokens, start_date):
        """Allocate monthly token budget to team"""
        budget_key = f"budget:{team}:{start_date.strftime('%Y-%m')}"

        budget = {
            'team': team,
            'allocated': monthly_tokens,
            'used': 0,
            'remaining': monthly_tokens,
```

```
            'start_date': start_date.isoformat(),
            'alerts_sent': []
        }

        self.redis.set(budget_key, json.dumps(budget))
        return budget

    def deduct_tokens(self, team, tokens):
        """Deduct tokens from team budget"""
        month = datetime.now().strftime('%Y-%m')
        budget_key = f"budget:{team}:{month}"

        budget_data = self.redis.get(budget_key)
        if not budget_data:
            raise Exception(f"No budget allocated for {team}")

        budget = json.loads(budget_data)
        budget['used'] += tokens
        budget['remaining'] = budget['allocated'] - budget['used']

        # Check thresholds and send alerts
        usage_percent = (budget['used'] / budget['allocated']) * 100

        if usage_percent >= 90 and '90%' not in budget['alerts_sent']:
            self.send_alert(team, usage_percent, budget)
            budget['alerts_sent'].append('90%')
        elif usage_percent >= 75 and '75%' not in budget['alerts_sent']:
            self.send_alert(team, usage_percent, budget)
            budget['alerts_sent'].append('75%')

        self.redis.set(budget_key, json.dumps(budget))

        return budget

    def can_use_tokens(self, team, tokens_needed):
        """Check if team has enough tokens"""
        month = datetime.now().strftime('%Y-%m')
        budget_key = f"budget:{team}:{month}"

        budget_data = self.redis.get(budget_key)
        if not budget_data:
            return False, "No budget allocated"

        budget = json.loads(budget_data)

        if budget['remaining'] < tokens_needed:
            return False, f"Insufficient tokens: {budget['remaining']} < {tokens_needed}"

        return True, "OK"

    def send_alert(self, team, usage_percent, budget):
        """Send alert when threshold reached"""
        print(f"""
        ■■  ALERT: Team '{team}' token usage at {usage_percent:.1f}%

        Used: {budget['used']:,} tokens
        Allocated: {budget['allocated']:,} tokens
        Remaining: {budget['remaining']:,} tokens

        Consider requesting additional budget or optimizing usage.
        """)

    def get_all_budgets(self):
        """Get budgets for all teams"""
        month = datetime.now().strftime('%Y-%m')
        pattern = f"budget:*:{month}"

        budgets = []
        for key in self.redis.scan_iter(pattern):
            budget_data = self.redis.get(key)
            if budget_data:
                budgets.append(json.loads(budget_data))

        return budgets

# Usage
budget_manager = TokenBudgetManager(redis_client)
```

```
# Allocate budgets
budget_manager.allocate_budget('engineering', 10_000_000, datetime.now())
budget_manager.allocate_budget('product', 5_000_000, datetime.now())
budget_manager.allocate_budget('marketing', 1_000_000, datetime.now())

# Check before using
can_use, message = budget_manager.can_use_tokens('engineering', 1000)
if can_use:
    # Use tokens
    budget_manager.deduct_tokens('engineering', 1000)
```

C. Priority Queue System

```
import heapq
from collections import defaultdict
import threading

class PriorityQueueManager:
    """Manage requests with priority queuing"""

    PRIORITIES = {
        'critical': 0,  # Highest priority
        'high': 1,
        'medium': 2,
        'low': 3,
        'batch': 4      # Lowest priority
    }

    def __init__(self):
        self.queues = defaultdict(list)
        self.lock = threading.Lock()
        self.counter = 0

    def enqueue(self, team, request_data, priority='medium'):
        """Add request to priority queue"""
        with self.lock:
            priority_value = self.PRIORITIES.get(priority, 2)
            # Use counter for FIFO within same priority
            heapq.heappush(
                self.queues[priority_value],
                (self.counter, team, request_data)
            )
            self.counter += 1

    def dequeue(self):
        """Get highest priority request"""
        with self.lock:
            # Check each priority level
            for priority in sorted(self.queues.keys()):
                if self.queues[priority]:
                    counter, team, request_data = heapq.heappop(self.queues[priority])
                    return team, request_data
        return None, None

    def get_queue_stats(self):
        """Get queue statistics"""
        with self.lock:
            stats = {}
            for priority, queue in self.queues.items():
                priority_name = [k for k, v in self.PRIORITIES.items() if v == priority][0]
                stats[priority_name] = len(queue)
            return stats

# Usage
queue_manager = PriorityQueueManager()

# High priority request (production)
queue_manager.enqueue('engineering', {'prompt': 'urgent'}, priority='high')

# Low priority request (testing)
queue_manager.enqueue('marketing', {'prompt': 'test'}, priority='low')
```

```
# Process requests by priority
team, request = queue_manager.dequeue()  # Gets 'engineering' first
```

2. Monitoring Dashboard

```
from flask import Flask, render_template_string
import json

app = Flask(__name__)

DASHBOARD_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
    <title>LLM Usage Dashboard</title>
    <style>
        body { font-family: Arial; margin: 20px; }
        .team-card {
            border: 1px solid #ddd;
            padding: 20px;
            margin: 10px 0;
            border-radius: 8px;
        }
        .usage-bar {
            width: 100%;
            height: 30px;
            background: #f0f0f0;
            border-radius: 5px;
            overflow: hidden;
        }
        .usage-fill {
            height: 100%;
            background: #4CAF50;
            transition: width 0.3s;
        }
        .usage-fill.warning { background: #FF9800; }
        .usage-fill.critical { background: #f44336; }
        .stats { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; }
        .stat-box { background: #f5f5f5; padding: 10px; border-radius: 5px; }
    </style>
</head>
<body>
    <h1>■ LLM Usage Dashboard</h1>

    {% for team, data in teams.items() %}
    <div class="team-card">
        <h2>{{ team|upper }}</h2>

        <div class="stats">
            <div class="stat-box">
                <strong>Tokens Used</strong><br>
                {{ "{:,}".format(data.tokens_used) }} / {{ "{:,}".format(data.tokens_limit) }}
            </div>
            <div class="stat-box">
                <strong>Cost</strong><br>
                ${{ "%.2f"|format(data.cost) }}
            </div>
            <div class="stat-box">
                <strong>Requests Today</strong><br>
                {{ data.requests_today }}
            </div>
        </div>

        <br>
        <strong>Usage: {{ "%.1f"|format(data.usage_percent) }}%</strong>
        <div class="usage-bar">
            <div class="usage-fill {% if data.usage_percent >= 90 %}critical{% elif data.usage
                 style="width: {{ data.usage_percent }}%"></div>
        </div>
    </div>
    {% endfor %}
```

```
    <script>
        // Auto-refresh every 30 seconds
        setTimeout(() => location.reload(), 30000);
    </script>
</body>
</html>
"""

@app.route('/dashboard')
def dashboard():
    """Usage dashboard"""
    teams_data = {}

    for team in ['engineering', 'product', 'marketing']:
        usage = get_usage_info(team)
        teams_data[team] = usage

    return render_template_string(DASHBOARD_TEMPLATE, teams=teams_data)
```

3. Cost Management

```
class CostManager:
    """Track and manage costs per team"""

    PRICING = {
        'gpt-4': {'input': 0.03, 'output': 0.06},        # per 1K tokens
        'gpt-3.5-turbo': {'input': 0.0015, 'output': 0.002},
        'claude-3-opus': {'input': 0.015, 'output': 0.075},
        'claude-3-sonnet': {'input': 0.003, 'output': 0.015}
    }

    def __init__(self, redis_client):
        self.redis = redis_client

    def calculate_cost(self, model, input_tokens, output_tokens):
        """Calculate cost for request"""
        pricing = self.PRICING.get(model, {'input': 0.01, 'output': 0.01})

        input_cost = (input_tokens / 1000) * pricing['input']
        output_cost = (output_tokens / 1000) * pricing['output']

        return input_cost + output_cost

    def track_cost(self, team, model, input_tokens, output_tokens):
        """Track cost for team"""
        cost = self.calculate_cost(model, input_tokens, output_tokens)

        # Monthly cost
        month = datetime.now().strftime('%Y-%m')
        cost_key = f"cost:{team}:{month}"
        self.redis.incrbyfloat(cost_key, cost)

        # Cost by model
        model_cost_key = f"cost:{team}:{month}:{model}"
        self.redis.incrbyfloat(model_cost_key, cost)

        return cost

    def get_cost_report(self, team, month=None):
        """Get cost report for team"""
        if not month:
            month = datetime.now().strftime('%Y-%m')

        cost_key = f"cost:{team}:{month}"
        total_cost = float(self.redis.get(cost_key) or 0)

        # Cost by model
        model_costs = {}
        for model in self.PRICING.keys():
            model_cost_key = f"cost:{team}:{month}:{model}"
            cost = float(self.redis.get(model_cost_key) or 0)
```

```
            if cost > 0:
                model_costs[model] = cost

        return {
            'team': team,
            'month': month,
            'total_cost': round(total_cost, 2),
            'by_model': model_costs
        }

    def set_budget_alert(self, team, budget_limit):
        """Set alert when budget limit reached"""
        month = datetime.now().strftime('%Y-%m')
        cost_key = f"cost:{team}:{month}"
        current_cost = float(self.redis.get(cost_key) or 0)

        if current_cost >= budget_limit:
            self.send_budget_alert(team, current_cost, budget_limit)

    def send_budget_alert(self, team, current_cost, budget_limit):
        """Send budget alert"""
        print(f"""
        ■ BUDGET ALERT: Team '{team}'

        Current cost: ${current_cost:.2f}
        Budget limit: ${budget_limit:.2f}
        Over budget: ${current_cost - budget_limit:.2f}

        Action required: Reduce usage or request budget increase.
        """)

# Usage
cost_manager = CostManager(redis_client)

# Track cost
cost = cost_manager.track_cost('engineering', 'gpt-4', 1000, 500)
print(f"Cost: ${cost:.4f}")

# Get report
report = cost_manager.get_cost_report('engineering')
print(f"Total cost: ${report['total_cost']}")
```

4. Policy & Governance

A. Usage Tiers

```
USAGE_TIERS = {
    'tier_1_critical': {
        'teams': ['engineering', 'product'],
        'requests_per_minute': 100,
        'tokens_per_month': 10_000_000,
        'models': ['gpt-4', 'claude-3-opus'],
        'priority': 'high',
        'approval_required': False
    },
    'tier_2_standard': {
        'teams': ['marketing', 'sales', 'support'],
        'requests_per_minute': 50,
        'tokens_per_month': 5_000_000,
        'models': ['gpt-3.5-turbo', 'claude-3-sonnet'],
        'priority': 'medium',
        'approval_required': False
    },
    'tier_3_experimental': {
        'teams': ['research', 'qa'],
        'requests_per_minute': 20,
        'tokens_per_month': 1_000_000,
        'models': ['gpt-3.5-turbo'],
```

```
        'priority': 'low',
        'approval_required': True
    }
}

def get_team_tier(team):
    """Get tier for team"""
    for tier_name, tier_config in USAGE_TIERS.items():
        if team in tier_config['teams']:
            return tier_name, tier_config
    return None, None
```

B. Approval Workflow

```
class ApprovalWorkflow:
    """Manage approval workflow for additional budget"""

    def __init__(self, redis_client):
        self.redis = redis_client

    def request_additional_budget(self, team, additional_tokens, justification):
        """Request additional token budget"""
        request_id = f"req_{team}_{int(time.time())}"

        request_data = {
            'request_id': request_id,
            'team': team,
            'additional_tokens': additional_tokens,
            'justification': justification,
            'status': 'pending',
            'requested_at': datetime.now().isoformat(),
            'approved_by': None,
            'approved_at': None
        }

        # Store request
        self.redis.set(f"approval:{request_id}", json.dumps(request_data))

        # Notify approvers
        self.notify_approvers(request_data)

        return request_id

    def approve_request(self, request_id, approver):
        """Approve budget request"""
        request_data = self.redis.get(f"approval:{request_id}")
        if not request_data:
            return False, "Request not found"

        request = json.loads(request_data)
        request['status'] = 'approved'
        request['approved_by'] = approver
        request['approved_at'] = datetime.now().isoformat()

        # Update budget
        month = datetime.now().strftime('%Y-%m')
        budget_key = f"budget:{request['team']}:{month}"
        budget_data = self.redis.get(budget_key)

        if budget_data:
            budget = json.loads(budget_data)
            budget['allocated'] += request['additional_tokens']
            budget['remaining'] += request['additional_tokens']
            self.redis.set(budget_key, json.dumps(budget))

        # Save approval
        self.redis.set(f"approval:{request_id}", json.dumps(request))

        # Notify team
        self.notify_team(request)
```

```
        return True, "Approved"

    def reject_request(self, request_id, approver, reason):
        """Reject budget request"""
        request_data = self.redis.get(f"approval:{request_id}")
        if not request_data:
            return False, "Request not found"

        request = json.loads(request_data)
        request['status'] = 'rejected'
        request['rejected_by'] = approver
        request['rejected_at'] = datetime.now().isoformat()
        request['rejection_reason'] = reason

        self.redis.set(f"approval:{request_id}", json.dumps(request))
        self.notify_team(request)

        return True, "Rejected"

    def notify_approvers(self, request):
        """Notify approvers of new request"""
        print(f"""
        ■ New Budget Request

        Team: {request['team']}
        Additional tokens: {request['additional_tokens']:,}
        Justification: {request['justification']}

        Review at: /admin/approvals/{request['request_id']}
        """)

    def notify_team(self, request):
        """Notify team of approval decision"""
        status_emoji = '■' if request['status'] == 'approved' else '■'
        print(f"""
        {status_emoji} Budget Request {request['status'].upper()}

        Team: {request['team']}
        Request ID: {request['request_id']}
        Status: {request['status']}
        """)

# Usage
workflow = ApprovalWorkflow(redis_client)

# Team requests more budget
request_id = workflow.request_additional_budget(
    team='marketing',
    additional_tokens=500_000,
    justification='New campaign launch, need extra capacity'
)

# Manager approves
workflow.approve_request(request_id, approver='john.doe@company.com')
```

C. Usage Policies

```
USAGE_POLICIES = {
    'allowed_use_cases': [
        'Product features',
        'Customer support',
        'Content generation',
        'Code assistance',
        'Data analysis'
    ],

    'prohibited_use_cases': [
        'Personal use',
        'External client work (without approval)',
        'Training competing models',
        'Generating harmful content',
```

```
            'Spam generation'
        ],

        'data_handling': {
            'pii_allowed': False,
            'customer_data': 'requires_approval',
            'proprietary_data': 'requires_legal_review',
            'retention_days': 30
        },

        'model_selection': {
            'default': 'gpt-3.5-turbo',  # Cost-effective
            'requires_approval': ['gpt-4', 'claude-3-opus'],  # Expensive
            'allowed_without_approval': ['gpt-3.5-turbo', 'claude-3-sonnet']
        }
    }

    def check_policy_compliance(team, use_case, model, contains_pii):
        """Check if request complies with policies"""

        checks = []

        # Use case check
        if use_case in USAGE_POLICIES['prohibited_use_cases']:
            checks.append({
                'check': 'use_case',
                'passed': False,
                'reason': f"Prohibited use case: {use_case}"
            })

        # PII check
        if contains_pii and not USAGE_POLICIES['data_handling']['pii_allowed']:
            checks.append({
                'check': 'pii',
                'passed': False,
                'reason': 'PII not allowed in requests'
            })

        # Model check
        if model in USAGE_POLICIES['model_selection']['requires_approval']:
            checks.append({
                'check': 'model',
                'passed': False,
                'reason': f"Model {model} requires approval"
            })

        all_passed = len(checks) == 0 or all(c['passed'] for c in checks)

        return all_passed, checks
```

5. Complete Implementation

Integrated Gateway

```
from flask import Flask, request, jsonify
import redis
from functools import wraps

app = Flask(__name__)
redis_client = redis.Redis(decode_responses=True)

# Initialize managers
budget_manager = TokenBudgetManager(redis_client)
cost_manager = CostManager(redis_client)
workflow = ApprovalWorkflow(redis_client)

@app.route('/api/v1/chat', methods=['POST'])
def chat_endpoint():
```

```
    """Main LLM endpoint with all controls"""

    # 1. Authentication
    api_key = request.headers.get('X-API-Key')
    team = get_team_from_api_key(api_key)
    if not team:
        return jsonify({'error': 'Invalid API key'}), 401

    data = request.json
    model = data.get('model', 'gpt-3.5-turbo')
    use_case = data.get('use_case', 'general')

    # 2. Rate limiting
    allowed, message = check_rate_limit(team)
    if not allowed:
        return jsonify({'error': 'Rate limit exceeded', 'message': message}), 429

    # 3. Policy compliance
    contains_pii = check_for_pii(data.get('messages', []))
    compliant, checks = check_policy_compliance(team, use_case, model, contains_pii)
    if not compliant:
        return jsonify({'error': 'Policy violation', 'checks': checks}), 403

    # 4. Token budget check
    estimated_tokens = estimate_tokens(data)
    can_use, msg = budget_manager.can_use_tokens(team, estimated_tokens)
    if not can_use:
        return jsonify({'error': 'Budget exceeded', 'message': msg}), 429

    # 5. Call LLM
    try:
        response = call_actual_llm(data)

        # 6. Track usage
        input_tokens = response['usage']['prompt_tokens']
        output_tokens = response['usage']['completion_tokens']
        total_tokens = input_tokens + output_tokens

        budget_manager.deduct_tokens(team, total_tokens)
        cost = cost_manager.track_cost(team, model, input_tokens, output_tokens)

        # 7. Add metadata
        response['metadata'] = {
            'team': team,
            'cost': round(cost, 4),
            'tokens_remaining': budget_manager.get_budget(team)['remaining']
        }

        return jsonify(response)

    except Exception as e:
        return jsonify({'error': str(e)}), 500

def estimate_tokens(data):
    """Estimate tokens for request"""
    # Rough estimate: 1 token ≈ 4 characters
    messages = data.get('messages', [])
    text = ' '.join([m.get('content', '') for m in messages])
    return len(text) // 4 + data.get('max_tokens', 1000)

def check_for_pii(messages):
    """Check if messages contain PII"""
    # Simplified - use actual PII detection
    import re

    for message in messages:
        content = message.get('content', '')
        # Check for email, SSN, etc.
        if re.search(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', content):
            return True
        if re.search(r'\b\d{3}-\d{2}-\d{4}\b', content):  # SSN
            return True

    return False

if __name__ == '__main__':
    app.run(debug=True, port=8000)
```

6. Admin Interface

```
@app.route('/admin/teams', methods=['GET'])
def admin_teams():
    """Admin view of all teams"""
    teams = []

    for team in ['engineering', 'product', 'marketing']:
        usage = get_usage_info(team)
        tier, config = get_team_tier(team)

        teams.append({
            'team': team,
            'tier': tier,
            'usage': usage,
            'config': config
        })

    return jsonify({'teams': teams})

@app.route('/admin/approvals', methods=['GET'])
def admin_approvals():
    """View pending approvals"""
    pattern = "approval:*"
    approvals = []

    for key in redis_client.scan_iter(pattern):
        approval_data = redis_client.get(key)
        if approval_data:
            approval = json.loads(approval_data)
            if approval['status'] == 'pending':
                approvals.append(approval)

    return jsonify({'approvals': approvals})

@app.route('/admin/approvals/<request_id>/approve', methods=['POST'])
def admin_approve(request_id):
    """Approve budget request"""
    approver = request.json.get('approver')
    success, message = workflow.approve_request(request_id, approver)

    return jsonify({'success': success, 'message': message})
```

Interview Answer (Complete)

Question: *"How do you limit usage of LLM across teams?"*

Answer:

"I implement a multi-layered governance system:

1. Technical Controls:
- **API Gateway**: All teams go through a central gateway with unique API keys
- **Rate Limiting**: Requests per minute/day limits per team (e.g., engineering: 100/min, marketing: 20/min)
- **Token Budgets**: Monthly token allocations (e.g., 10M for engineering, 1M for marketing)
- **Priority Queuing**: Critical teams get priority access during high load

2. Cost Management:
- **Real-time Tracking**: Track costs per team, per model
- **Budget Alerts**: Alert at 75% and 90% usage thresholds
- **Cost Attribution**: Detailed breakdown by team and model
- **Approval Workflow**: Teams can request additional budget with justification

3. Usage Policies:
- **Tiered Access**: Different tiers with different limits and model access
- **Policy Compliance**: Enforce allowed use cases, prohibit personal use
- **Model Selection**: Default to cost-effective models (GPT-3.5), require approval for expensive ones (GPT-4)
- **Data Handling**: No PII allowed, customer data requires approval

4. Monitoring & Visibility:
- **Dashboard**: Real-time usage dashboard showing all teams
- **Alerts**: Automatic alerts when limits approached
- **Reports**: Monthly cost reports per team
- **Audit Logs**: Track all requests for compliance

Implementation Example:

```
# Team makes request
response = llm_gateway.call(
    api_key='team_key',
    model='gpt-4',
    prompt='...'
)

# Gateway checks:
# 1. Rate limit (100 req/min) ■
# 2. Token budget (10M/month) ■
# 3. Policy compliance ■
# 4. Cost tracking ■
```

Benefits:
- Fair resource allocation
- Cost control (saved 40% in first quarter)
- Prevents abuse
- Visibility into usage patterns
- Easy to scale as org grows"

Summary Checklist

Technical Implementation ■

[ ] API Gateway with authentication [ ] Rate limiting (per minute, per day) [ ] Token budget system (monthly allocations) [ ] Priority queuing for critical teams [ ] Cost tracking per team/model [ ] Usage monitoring dashboard Policy & Governance ■

[ ] Usage tiers defined [ ] Allowed/prohibited use cases documented [ ] Model selection policy [ ] Data handling rules (PII, customer data) [ ] Approval workflow for exceptions Monitoring & Alerts ■

[ ] Real-time dashboard [ ] Budget threshold alerts (75%, 90%) [ ] Cost anomaly detection [ ] Usage trend reports [ ] Audit logs for compliance Administration ■

[ ] Admin interface for team management [ ] Budget allocation tools [ ] Approval workflow [ ] Usage reports [ ] Team onboarding process Example Metrics

Before Implementation:
- Uncontrolled spending: $50K/month
- No visibility into who uses what
- Teams blocking each other
- No way to attribute costs

After Implementation:
- Controlled spending: $30K/month (40% savings)
- Full visibility per team
- Fair resource allocation
- Accurate cost attribution
- Teams can scale within budgets

Code Repository Structure

```
llm-governance/
■■■ gateway/
■   ■■■ api_gateway.py
■   ■■■ rate_limiter.py
■   ■■■ token_budget.py
■   ■■■ priority_queue.py
■■■ monitoring/
■   ■■■ dashboard.py
■   ■■■ cost_tracker.py
■   ■■■ alerts.py
■■■ admin/
■   ■■■ team_management.py
■   ■■■ approval_workflow.py
■   ■■■ reports.py
■■■ policies/
■   ■■■ usage_policies.py
■   ■■■ compliance_checker.py
■■■ README.md
```

Next Steps

Deploy API Gateway - Central entry point Set Team Quotas - Define limits per team Configure Monitoring - Dashboard + alerts Document Policies - Usage guidelines Train Teams - Onboarding process Review Monthly - Adjust quotas based on usage Related Questions You Might Get

Q: "What if a team needs more budget mid-month?"
A: "We have an approval workflow. Team submits request with justification, manager reviews usage patterns, and can approve additional allocation. We track these exceptions to refine next month's budget."

Q: "How do you handle urgent requests when quota is exhausted?"
A: "We have a priority queue system. Critical teams (like production systems) have high priority. They can also have an emergency reserve (10% of monthly budget) that requires approval to use."

Q: "What about teams gaming the system?"
A: "We have multiple safeguards: (1) Audit logs track all requests, (2) Policy compliance checks prevent prohibited use cases, (3) Anomaly detection alerts unusual patterns, (4) Regular reviews with team leads."

Q: "How do you allocate budgets fairly?"
A: "Based on three factors: (1) Business criticality (production > experimentation), (2) Historical usage patterns, (3) ROI of use cases. We review quarterly and adjust based on actual needs and business value delivered."

Tools & Technologies

Implementation:
- **API Gateway**: Flask/FastAPI with Redis
- **Rate Limiting**: Redis with sliding window
- **Monitoring**: Prometheus + Grafana
- **Dashboard**: Flask + Chart.js
- **Alerting**: PagerDuty/Slack
- **Storage**: Redis (hot data), PostgreSQL (historical)

Alternative Solutions:
- **Open Source**: Kong Gateway, Tyk
- **Cloud**: AWS API Gateway, Azure API Management
- **SaaS**: LangSmith, Helicone, Portkey

Key Takeaways

Multi-layered approach (technical + policy + monitoring) **Fair allocation** (tiers based on criticality) **Cost control** (budgets + alerts) **Flexibility** (approval workflow for exceptions) **Visibility** (dashboard + reports) **Bottom line:** Governance is about enabling teams to use LLMs effectively while maintaining cost control and compliance.